DJAMA PRO

AUDIT COMPLET

Module Sourcing IA

Expert Sourcing & Marchés IA · Fournisseurs · Marchés · Analyse Sonnet

Audit de sécurité, architecture, capacités IA et recommandations
DJAMA PRO v2 — Next.js 16 · React 19 · Supabase · Anthropic Claude

Produit le 3 octobre 2026 · Claude Code / AMDJAD Nofane

Routes API auditées	5
Pages frontend auditées	3
Tables Supabase	2 (+ ERP)
Modèle IA principal	claude-sonnet-4-5
Capacités fonctionnelles	18 / 26
Lacunes sécurité critiques	3

SECTION 01

Objectif actuel du module Sourcing IA

Le module Sourcing IA est un outil d'aide à l'approvisionnement B2B intégré à DJAMA PRO. Il se présente comme un « Expert Sourcing & Marchés IA » et propose trois espaces distincts : un **Chat IA** alimenté par Claude Sonnet pour des conseils stratégiques, un wizard **Fournisseurs** en 6 étapes pour rechercher, comparer et contacter des fournisseurs internationaux, et un wizard **Appel d'offre** en 6 étapes pour analyser des marchés publics et générer des dossiers de candidature. Un onglet **Négociations** permet un suivi CRUD des négociations en cours.

Structure de navigation

Route	Titre UI	Objectif
/client/sourcing	Sourcing IA — Chat + Négociations	Chat stratégique + suivi négos
/client/sourcing/fournisseurs	Recherche Fournisseurs	Wizard 6 étapes : recherche → documents
/client/sourcing/appel-offre	Appel d'offre	Wizard 6 étapes : analyse → documents AO

Ce que l'interface promet vs ce qui est implémenté

L'interface présente le module comme un outil complet couvrant la recherche de fournisseurs mondiaux, les marchés publics et privés, la négociation, la rédaction de documents et l'analyse de risques. L'ensemble de ces fonctionnalités est réellement implémenté — mais avec des nuances importantes détaillées dans les sections suivantes (notamment : les fournisseurs sont générés par IA, les PDFs ne sont pas analysés dans l'étape d'analyse d'AO, et les liens email/WhatsApp sont incomplets).

SECTION 02

Cartographie complète du code

Fichiers identifiés

Catégorie	Chemin	Rôle
Page	src/app/client/sourcing/page.tsx	Chat IA + Négociations (1 273 lignes)
Page	src/app/client/sourcing/fournisseurs/page.tsx	Wizard fournisseurs (1 637 lignes)
Page	src/app/client/sourcing/appel-offre/page.tsx	Wizard appel d'offre (1 322 lignes)
API	src/app/api/sourcing/chat/route.ts	Chat IA Claude Sonnet
API	src/app/api/sourcing/fournisseurs/search/route.ts	Recherche fournisseurs Claude Haiku
API	src/app/api/sourcing/fournisseurs/generate/route.ts	Génération 11 documents fournisseur

Catégorie	Chemin	Rôle
API	src/app/api/sourcing/appel-offre/analyze/route.ts	Analyse AO Claude Sonnet
API	src/app/api/sourcing/appel-offre/generate/route.ts	Génération 8 documents AO + PDF multimodal
Lib	src/lib/sourcing/generateGuide.ts	Guide sourcing (helper)
Lib	src/lib/sourcing-pdf.ts	Export PDF (jsPDF)
SQL	supabase/migrations/upgrade_to_5star.sql	Tables sourcing_sessions + sourcing_negotiations
SQL	supabase/migrations/061_fournisseurs_p0.sql	Tables ERP fournisseur (séparé)

Schéma de flux : Interface → API → IA → DB → Modules DJAMA

- **Interface utilisateur** → 3 pages Next.js « use client »
- ↓ **fetch()** vers les 5 routes API Route Handler (Next.js App Router)
- **API routes** → auth Supabase SSR → rate limit → Anthropic SDK
- **Anthropic Claude** → claude-sonnet-4-5 (Chat, Generate, AO) ou claude-haiku-4-5-20251001 (Search)
- ↓ **résultats IA** parsés côté serveur, retournés au frontend
- **Supabase client frontend** → sourcing_sessions (historique), sourcing_negotiations (CRUD direct)
- **Intégration sortante** → table contacts (export CRM fournisseurs)
- **Intégration ERP** → fournisseur_orders + fournisseur_invoices → expenses (via trigger)

Note : Le module Sourcing IA n'est PAS connecté aux tables fournisseurs ERP (fournisseur_orders, fournisseur_invoices) par le code Sourcing — ces tables appartiennent au module Fournisseurs ERP séparé. La seule connexion inter-modules documentée est l'export CRM (fournisseurs → contacts).

SECTION 03

Audit du Chat IA

Capacité	État	Détail
Envoi de message	VÉRIFIÉ DANS LE CODE	fetch POST /api/sourcing/chat avec message + history
Réponse structurée JSON	VÉRIFIÉ DANS LE CODE	text + reasoning + sections[] + actions + suggestions
Historique de session	VÉRIFIÉ DANS LE CODE	20 dernières sessions depuis sourcing_sessions
Persistance	VÉRIFIÉ DANS LE CODE	sourcing_sessions.messages (jsonb) sur newChat()
Contexte multi-tours	VÉRIFIÉ DANS LE CODE	8 derniers tours injectés dans le prompt
Streaming	ABSENT	API non-streaming, réponse complète en one-shot
Pièces jointes	ABSENT	Aucune gestion de fichier dans le Chat IA principal
Mémoire persistante	PARTIEL	Sessions sauvegardées mais non rechargées dans le chat actif
Suppression session	ABSENT	Aucun bouton de suppression de session dans l'UI
Retry automatique	ABSENT	maxRetries=1 sur l'API, mais pas de retry côté frontend

Capacité	État	Détail
Actions structurées DJAMA	ABSENT	L'IA ne peut pas modifier la DB — chat consultatif uniquement
Export PDF conversation	VÉRIFIÉ DANS LE CODE	jsPDF avec design rapport + sections visuelles
Export CRM	VÉRIFIÉ DANS LE CODE	Insère fournisseurs dans contacts avec organization_id
Suggestions de suivi	VÉRIFIÉ DANS LE CODE	5 suggestions générées par l'IA après chaque réponse
Rate limiting	PARTIEL	20 req/h mais Map in-memory non distribué (risque multi-instances

Le Chat IA est un assistant consultatif, pas un agent actif. Il génère des recommandations, analyses et documents textuels mais ne peut effectuer aucune action dans DJAMA (créer un devis, modifier un fournisseur, déclencher une commande). Les réponses sont des JSON structurés avec raisonnement stratégique et sections typées (supplier_list, steps, checklist, tips).

SECTION 04

Intelligence artificielle — configuration détaillée

Route	Modèle	max_tokens	Timeout	Retries	Streamir
/sourcing/chat	claude-sonnet-4-5	4 096	90 s	1	Non
/fournisseurs/search	claude-haiku-4-5-20251001	4 500	50 s	0	Non
/fournisseurs/generate	claude-sonnet-4-5	2 000/doc	100 s	1	Non
/appel-offre/analyze	claude-sonnet-4-5	4 096	110 s	1	Non
/appel-offre/generate	claude-sonnet-4-5	2 000/doc	90 s	0	Non

Observations clés

- **Provider :** Anthropic uniquement, via @anthropic-ai/sdk. Pas d'abstraction multi-provider.
- **Claude Sonnet 4-5** pour toutes les routes sauf la recherche (Haiku). L'interface affiche bien « Analyse Sonnet » — VÉRIFIÉ DANS LE CODE.
- **Format JSON forcé** dans le prompt du chat IA. Le serveur parse le JSON et retourne une structure typée. En cas d'échec de parsing, un fallback texte brut est retourné.
- **Clé API :** process.env.ANTHROPIC_API_KEY — jamais exposée côté client. VÉRIFIÉ DANS LE CODE.
- **Aucun suivi de coûts** — pas de comptage tokens, pas de limite de budget par organisation.
- **Outils/functions Claude :** ABSENT — pas de tool_use, l'IA répond uniquement en texte/JSON.
- **Temperature :** non configurée (valeur par défaut Anthropic : 1.0).
- **PDF multimodal :** /appel-offre/generate envoie des PDFs en base64 via l'API beta Claude (type: 'document').

SECTION 05

Capacités Sourcing — état réel

Capacité	État	Preuve dans le code
Rechercher des fournisseurs	FONCTIONNEL	/fournisseurs/search → 5 fournisseurs + marché + logistique
Comparer fournisseurs	FONCTIONNEL	ComparisonMatrix (2+ suppliers) + vue analytics fournisseurs/page.tsx
Analyser des offres	FONCTIONNEL	/fournisseurs/generate → stratégie négociation + tableau comparatif
Analyser des prix	FONCTIONNEL	prix_unite, marge potentielle, analyse_marche dans SearchResult
Comparer MOQ	FONCTIONNEL	moq dans Supplier, tableau comparatif ASCII généré
Comparer délais	FONCTIONNEL	delai_fab + delai_transport dans Supplier et analytics
Comparer conditions paiement	PARTIEL	conditions_paiement dans les docs générés, non dans la structure SearchResult
Comparer Incoterms	PARTIEL	Dans doc contrat/stratégie négociation — non dans la structure de données
Analyser risques	FONCTIONNEL	risques[] par fournisseur + risques_globaux dans SearchResult
Préparer RFQ/RFP	FONCTIONNEL	rfq_fr + rfq_en générés (doc type 'rfq_fr' et 'rfq_en')
Rédiger demandes de devis	FONCTIONNEL	rfq_fr + rfq_en + whatsapp_fr + whatsapp_en générés
Recommander des questions	FONCTIONNEL	questions_fournisseur — 7 catégories min 5 questions
Créer tableaux comparatifs	FONCTIONNEL	tableau_comparatif ASCII généré par /fournisseurs/generate
Stratégie d'approvisionnement	FONCTIONNEL	doc 'negociation' : BATNA + tactiques + script de négociation
Marchés publics — analyse	FONCTIONNEL	/appel-offre/analyze → taux_succes + critères + recommandations
Marchés publics — documents	FONCTIONNEL	/appel-offre/generate → 8 docs (mémoire, DC1, DC2, etc.)
Analyse de documents PDF	PARTIEL	/analyze : PDFs non lus. /generate : PDFs envoyés en multimodal
Actions directes dans DJAMA	ABSENT	Aucun tool_use, aucune écriture DB depuis le chat IA
Calcul automatique TCO	ABSENT	Non implémenté — coûts logistiques estimés mais pas de TCO calculé
Suivi performance fournisseurs	ABSENT	Aucune connexion avec fournisseur_orders/invoices
Création fournisseur DJAMA ERP	ABSENT	Export CRM uniquement (table contacts, pas les tables ERP)
Veille marché temps réel	ABSENT	Données LLM uniquement — pas de web search ni API externe
Scoring fournisseur automatisé	PARTIEL	niveau_confiance 0-100 généré par IA, non persisté en DB
Gestion documents fournisseur	ABSENT	Upload 'joindre contrat' = state local uniquement, non sauvegardé
Intégration Dépenses/Compta	ABSENT	Aucun lien depuis Sourcing IA vers expenses/treasury
Intégration Stocks	ABSENT	Non implémenté

SECTION 06

Sources de données — internet, LLM, ou DB ?

Origine des données fournisseurs

Les fournisseurs retournés par Sourcing IA sont **générés par le LLM**, basés sur la connaissance interne du modèle Claude (entraîné jusqu'à début 2025). Il n'existe aucune recherche web en temps réel, aucun appel à des APIs externes (Alibaba, Kompass, ThomasNet, BOAMP...) ni aucune base de données fournisseurs interne à DJAMA.

Source	Utilisée ?	Détail
LLM (Claude) — connaissance modèle	OUI — PRINCIPAL	Tous les fournisseurs, prix, données marché
Base DJAMA (fournisseurs ERP)	NON	Aucune lecture des tables fournisseur ERP
Recherche web (Brave, Serper...)	NON	Aucune intégration de web search
APIs Alibaba / GlobalSources	NON	Les URLs générées sont des liens de recherche, pas des APIs
APIs marchés publics (BOAMP, TED)	NON	Aucune intégration directe
Fichiers uploadés (PDF/DOCX)	PARTIEL	/analyze ne lit pas les PDFs ; /generate les envoie en multimodal
Contexte historique sessions	OUI (chat)	8 derniers tours de conversation injectés dans le prompt

Risques d'hallucination — analyse

Risque élevé pour : Noms et adresses de fournisseurs (peuvent être fictifs), numéros de certification ISO/CE spécifiques, prix unitaires exacts, MOQ précis, délais de livraison, URLs de sites web fournisseurs. Le système prompt demande des « données réalistes et précises » mais sans mécanisme de vérification.

Risque moyen pour : Données logistiques (fourchettes de prix fret, délais, taux de droits de douane), analyse de marché (tendances générales), plateformes de recherche (Alibaba, Made-in-China — les plateformes existent mais les URLs de recherche générées peuvent ne pas retourner de résultats pertinents).

Ce que l'IA fait bien : Structuration des données, analyse comparative, rédaction de documents professionnels, couverture des spécificités réglementaires par territoire (DROM-COM notamment), stratégie de négociation.

■■ Recommandation critique : Ajouter un avertissement visible indiquant que les fournisseurs sont générés par IA et doivent être vérifiés avant tout engagement commercial.

SECTION 07

Intégration module Fournisseurs

Action	État	Détail
Lire fournisseurs ERP existants	ABSENT	Aucune requête vers fournisseur_orders ou tables ERP

Action	État	Détail
Comparer plusieurs fournisseurs	FONCTIONNEL	ComparisonMatrix + analytics dans fournisseurs/page.tsx
Créer un fournisseur ERP	ABSENT	Export vers contacts uniquement (pas vers tables fournisseur ERP)
Mettre à jour un fournisseur	ABSENT	Non implémenté
Rattacher documents/offres	ABSENT	Upload 'joindre contrat' = state local non persisté
Consulter historique achats	ABSENT	Aucune connexion avec fournisseur_invoices
Analyser performances fournisseur	ABSENT	Aucune connexion avec les données ERP
Export CRM (table contacts)	FONCTIONNEL	supabase.from('contacts').insert() avec organization_id
Duplication base fournisseurs	NON	Pas de table fournisseurs propre à Sourcing IA (sourcing_sessions unique

Le module Sourcing IA ne duplique pas la base fournisseurs ERP, mais il ne la lit pas non plus. Les deux modules coexistent de façon totalement indépendante. L'unique pont est l'export CRM vers la table contacts.

SECTION 08

Audit de l'onglet Négociations

L'onglet Négociations est implémenté dans sourcing/page.tsx via le composant NegotiationsPanel. Il gère des négociations via CRUD Supabase direct depuis le frontend (sans route API dédiée).

Champ	Dans le code	Implémenté
Fournisseur (nom)	sourcing_negotiations.supplier TEXT NOT NULL	OUI
Statut (4 valeurs)	status TEXT (sans contrainte CHECK)	OUI — sans contrainte DB
Notes	notes TEXT DEFAULT ''	OUI
Montant	amount TEXT nullable	OUI (texte libre)
Date création/màj	created_at + updated_at timestamptz	OUI
Produit/service	ABSENT	NON
Prix initial/cible	ABSENT	NON
Quantité / MOQ	ABSENT	NON
Devise / Incoterm	ABSENT	NON
Transport / Délais	ABSENT	NON
Conditions paiement	ABSENT	NON
Messages / contre-offres	ABSENT	NON
Historique négociation	ABSENT	NON
Assistance IA active	ABSENT	NON — l'IA répond en chat mais ne pilote pas l

Champ	Dans le code	Implémenté
Résultat final documenté	ABSENT	NON

Verdict : L'onglet Négociations est **une coquille fonctionnelle minimale**. Il permet de noter qu'une négociation est « En cours » ou « Acceptée » avec un nom de fournisseur et des notes libres. La table ne contient que 7 colonnes. Il n'y a aucune structure pour les données commerciales clés (prix, délais, conditions), aucun fil de messages, aucune IA active. La stratégie de négociation IA est dans le wizard Fournisseurs (doc 'negociation'), pas dans cet onglet.

SECTION 09

L'onglet « Marchés »

L'interface principale (/client/sourcing) affiche 4 cartes de démarrage rapide (quickstart). La carte « Marchés » déclenche simplement une question prédéfinie dans le chat IA : « Identifie les marchés publics ouverts dans mon secteur d'activité et donne-moi les opportunités prioritaires à saisir ce trimestre ». Il n'existe PAS d'onglet ou de page dédiée aux Marchés — c'est uniquement un point d'entrée vers le Chat IA.

Fonctionnalité	État	Détail
Onglet dédié Marchés	ABSENT	Pas de page /client/sourcing/marches
Analyse marchés publics (AO)	FONCTIONNEL	Wizard /appel-offre — analyse + 8 docs
Veille BOAMP/TED en temps réel	ABSENT	LLM uniquement, pas d'API marchés publics
Marchés privés — opportunités	PARTIEL	Chat IA peut en parler, non structuré
Analyse de marché sectorielle	PARTIEL	Couvert par le chat IA — texte non structuré
Table Supabase marchés	ABSENT	Aucune table dédiée aux marchés

SECTION 10

Intégrations avec les autres modules DJAMA

Module DJAMA	État	Détail technique
Fournisseurs ERP	ABSENT	Aucune lecture fournisseur_orders / fournisseur_invoices
CRM (contacts)	FONCTIONNEL	Export fournisseurs → supabase.from('contacts').insert()
Dépenses	ABSENT	Aucun lien direct depuis Sourcing IA
Trésorerie	ABSENT	Non implémenté
Comptabilité	ABSENT	Non implémenté
Contrats	ABSENT	Documents générés en texte — non sauvegardés en DB
Stocks	ABSENT	Non implémenté

Module DJAMA	État	Détail technique
Projets	ABSENT	Non implémenté
AI Docs	ABSENT	Pas de lien entre documents générés et AI Docs
Scanner	ABSENT	Aucune connexion — upload PDF indépendant
Checklists	ABSENT	Non implémenté
Trigger ERP (trigger)	PARTIEL	fournisseur_invoices → expenses via trigger SQL (pas Sourcing IA)

Le module Sourcing IA est actuellement quasi-isolé du reste de DJAMA PRO. Sa seule intégration active est l'export CRM. Les documents générés (RFQ, contrats, stratégies) ne sont pas persistés dans la base DJAMA — ils existent uniquement en mémoire frontend jusqu'à l'export PDF.

SECTION 11

Gestion des documents

Type document	Upload	Lecture réelle	Stockage DB	Sécurité
PDF (appel-offre/analyze)	OUI	NON — nom/taille seulement	NON	Mémoire frontend uniquement
PDF (appel-offre/generate)	OUI	OUI — multimodal Claude API	NON	Transmis à l'API Anthropic
DOCX	OUI	OUI — texte extrait	NON	Mémoire frontend uniquement
TXT	OUI	OUI — content lu	NON	Mémoire frontend uniquement
Excel (.xlsx)	NON	NON	NON	Extension rejetée
Image	NON	NON	NON	Extension rejetée
Fiche technique	NON	NON	NON	Extension rejetée
Contrat fournisseur	UI	NON — state local uniquement	NON	Jamais transmis

Point critique : aucun document uploadé n'est sauvegardé en base de données ou dans Supabase Storage. Tout est en mémoire frontend. Si l'utilisateur recharge la page, tous les fichiers uploadés sont perdus. Le contenu transmis à Claude (pour /appel-offre/generate) transite par les serveurs Anthropic — l'utilisateur doit en être informé pour des documents confidentiels.

SECTION 12

Audit de sécurité

Points positifs

- **Authentification sur toutes les routes API :** supabase.auth.getUser() + 401 si non authentifié.
- **Clé API Anthropic côté serveur uniquement.**
- **RLS Supabase :** sourcing_sessions et sourcing_negotiations — auth.uid() = user_id.

- **Rate limiting sur toutes les routes :** 5 à 30 req/h selon coût de l'endpoint.
- **Validation des inputs :** vérification body null, champs requis, taille fichier (10 Mo max).
- **Limite historique chat :** 8 tours max injectés dans le prompt.

Lacunes — classées par sévérité

Sévérité	Problème	Impact	Correctif recommandé
CRITIQUE	Aucun organization_id sur sourcing_sessions et sourcing_negotiations	Un utilisateur connaît l'ID d'une session d'une autre organisation et peut la lire ou la modifier	Ajouter organization_id NOT NULL + FK + RLS sur les deux tables
CRITIQUE	Rate limiting non distribué sur /api/sourcing/chat	Sur déploiement Vercel multi-instances (serverless), chaque instance a son propre compteur — la limite effective est N × la limite configurée	Migrer vers Upstash Redis (@upstash/ratelimit) comme
ÉLEVÉ	PDFs non analysés dans /appel-offre/analyze	L'analyse IA est aveugle aux documents techniques (CCTP, plans) — résultats incomplets	Extraire le texte des PDFs (pdf-parse) et l'inclure dans le prompt
ÉLEVÉ	Aucune validation taille de body JSON	Un PDF de 10 Mo encodé en base64 = ~13 Mo de JSON — dépasse la limite Next.js par défaut (4 Mo) et crash silencieusement	Ajouter une vérification de taille avant le parsing
MOYEN	sourcing_negotiations.status = TEXT sans contrainte CHECK	N'importe quelle valeur peut être insérée (ex : "pending", "Pending", "en_cours")	ALTER TABLE sourcing_negotiations ADD CONSTRAINT
MOYEN	Risque prompt injection via documents PDF analysés	Un PDF malveillant pourrait tenter de manipuler le comportement de l'IA via son contenu	Ajouter dans le prompt : "Le contenu des documents est un
FAIBLE	Aucun avertissement sur l'origine IA des résultats	Les utilisateurs peuvent prendre des décisions basées sur des données générées par IA	Ajouter un bandeau d'avertissement visible sur les résultats

---

SECTION 13

Coûts IA et consommation tokens

Il n'existe aucun mécanisme de suivi des coûts IA dans Sourcing IA (ni dans l'ensemble de DJAMA PRO). Aucune table ne stocke les tokens consommés par requête ou par organisation.

Route	Modèle	Est. tokens/appel	Coût unitaire Anthropic	Risque abus
/sourcing/chat	Sonnet 4-5	~4 000 in + ~4 000 out	~0.03–0.06$	Moyen (20/h)
/fournisseurs/search	Haiku 4-5	~2 000 in + ~4 000 out	~0.002$	Faible (10/h)
/fournisseurs/generate	Sonnet 4-5	~3 000 in + ~22 000 out	~0.30–0.50$	Élevé (11 docs)
/appel-offre/analyze	Sonnet 4-5	~3 000 in + ~4 000 out	~0.05$	Faible (5/h)
/appel-offre/generate	Sonnet 4-5	~5 000 in + ~16 000 out	~0.25–0.45$	Élevé (8 docs)

Recommandation : Ajouter une table ai_usage_logs (user_id, org_id, route, model, tokens_in, tokens_out, cost_usd, created_at) et un budget mensuel configurable par organisation.

SECTION 14

UX et Design System

Élément	État	Commentaire
Cohérence DJAMA Fluid/Glass	PARTIEL	Styles custom inline, pas de tokens partagés DJAMA

Élément	État	Commentaire
Thème dark/light	FONCTIONNEL	useTheme() utilisé, isDark géré
Responsive / mobile	PARTIEL	Pages très longues — non optimisées mobile
Chat IA — zone saisie	FONCTIONNEL	Textarea + suggestions + envoi
Wizard 6 étapes (fournisseurs)	FONCTIONNEL	Stepper + animations framer-motion
Wizard 6 étapes (AO)	FONCTIONNEL	Idem, bien structuré
Espace central chat	À REPENSER	Grande zone vide avant le premier message
Onglet Négociations	MINIMAL	Fonctionnel mais pauvre (7 champs seulement)
Icônes	FONCTIONNEL	Lucide React (cohérent DJAMA)
Animations	FONCTIONNEL	Framer Motion v12 — loading, transitions
Export PDF	FONCTIONNEL	jsPDF — design rapport professionnel
Accessibilité	NON AUDITÉ	Pas d'audit WCAG réalisé

SECTION 15

Tests réels

Les tests suivants ne peuvent pas être exécutés dans cet audit car ils nécessitent un serveur Next.js en cours d'exécution, une clé API Anthropic active et une session Supabase authentifiée. L'analyse ci-dessous est basée sur la lecture du code source.

Test	Méthode vérification	Verdict basé sur le code	Verdict réel
A : Recherche 500 laptops	Lecture /fournisseurs/search	Retourne 5 fournisseurs LLM + données marché si disponibles	NON TESTÉ EN LIVE
B : Tableau comparatif	Lecture /fournisseurs/generate	Document 'tableau_comparatif' ASCII généré. Comparaison dans l'UI.	NON TESTÉ EN LIVE
C : Stratégie -8%	Lecture /fournisseurs/generate	Document 'negociation' avec BATNA + tactiques + script	NON TESTÉ EN LIVE
D : Demande de devis	Lecture /fournisseurs/generate	Documents rfq_fr + rfq_en générés par Claude Sonnet.	NON TESTÉ EN LIVE
E : Test négociation	Lecture sourcing/page.tsx	CRUD basique (nom + statut + notes + montant). Pas de structure de négociation.	NON TESTÉ EN LIVE

Pour des tests réels PASS/FAIL, l'environnement de développement doit être démarré (npm run dev) avec une clé ANTHROPIC_API_KEY valide dans .env.local et un compte DJAMA PRO actif.

SECTION 16

Rapport final et Roadmap

A — Ce que Sourcing IA fait réellement aujourd'hui

- Chat IA complet avec Claude Sonnet 4-5, réponses structurées JSON, export PDF
- Recherche de fournisseurs internationaux (données générées par LLM, adaptées au territoire)
- Génération de 11 types de documents fournisseur professionnels (RFQ, contrats, stratégie, etc.)
- Analyse d'appel d'offre avec score de succès et recommandations
- Génération de 8 documents de réponse AO conformes au droit français
- Analyse logistique par territoire (7 DROM-COM avec spécificités douanières)
- Vue analytics comparative des fournisseurs
- Suivi de négociations (CRUD minimal : 7 champs)
- Export CRM des fournisseurs vers la table contacts
- Historique des sessions de chat (20 dernières, persistées en Supabase)

B — Ce qui existe mais est incomplet

- Analyse AO : PDFs non lus dans /analyze (seuls DOCX et TXT sont analysés)
- Négociations : 7 champs seulement, aucune structure commerciale, aucune IA active
- Persistance documents : générés en mémoire frontend, non sauvegardés en DB
- Organization_id : absent des tables sourcing (données personnelles uniquement)
- Rate limiting chat : non distribué (Map in-memory vs Redis pour les autres routes)

C — Ce qui est uniquement visuel / décoratif

- Bouton 'Joindre contrat' sur les fiches fournisseurs (state local, non persisté)
- Bouton WhatsApp sans numéro de téléphone (lien générique wa.me sans destinataire)
- Bouton Email RFQ sans adresse email (mailto: sans destinataire)
- URLs fournisseurs : liens de recherche génériques, pas de profils réels
- Carte 'Marchés' : point d'entrée vers chat uniquement, pas de page/table dédiée

D — Ce qui est absent

- Connexion avec les tables fournisseurs ERP (fournisseur_orders, fournisseur_invoices)
- Veille marchés publics en temps réel (BOAMP, TED, UGAP)
- Web search / APIs fournisseurs réels
- Persistance des documents générés (Supabase Storage ou DB)
- Suivi des coûts IA / tokens consommés
- Onglet Marchés dédié avec données structurées
- Tool use / actions IA directes dans DJAMA (créer commande, modifier fournisseur)
- Streaming des réponses IA
- Intégration Stocks, Dépenses, Comptabilité, Contrats, Projets
- Scoring fournisseur persistant et évolutif

- Analyse TCO complète

E — Problèmes critiques

- CRITIQUE : Aucun organization_id → données non isolées par organisation
- CRITIQUE : Rate limiting non distribué sur /sourcing/chat
- ÉLEVÉ : Fausse promesse d'analyse PDF dans /appel-offre/analyze
- ÉLEVÉ : Aucun avertissement visible que les fournisseurs sont générés par IA
- ÉLEVÉ : Documents générés perdus au rechargement de page
- MOYEN : Risque prompt injection via PDFs uploadés

F — Sécurité

Authentification : ✓ 5/5 routes protégées. Rate limiting : ✓ 5/5 routes avec limites adaptées. Isolation utilisateur : ✓ RLS auth.uid(). Isolation organisation : ✗ Absent. Prompt injection : ✗ Non protégé (Scanner l'a fait, à répliquer ici).

G — Architecture actuelle

3 pages Next.js « use client » → 5 routes API App Router → Anthropic SDK (claude-sonnet-4-5 + claude-haiku) → 2 tables Supabase (sourcing_sessions, sourcing_negotiations) sans organization_id. Aucune connexion avec les autres modules DJAMA (sauf export CRM). Données fournisseurs générées par LLM, non persistées. Documents générés en mémoire frontend, exportables en PDF.

H — Architecture recommandée

- Ajouter organization_id aux 2 tables + migration + RLS
- Migrer /sourcing/chat vers checkRateLimitAsync (Upstash Redis)
- Persister les documents générés dans Supabase Storage + table sourcing_documents
- Corriger /appel-offre/analyze pour analyser réellement les PDFs (multimodal Claude)
- Ajouter table ai_usage_logs pour suivi coûts et quotas
- Connecter Sourcing IA aux tables fournisseur ERP (lecture) et stocks (lecture)
- Ajouter Claude tool_use pour actions directes (créer fournisseur, créer commande)
- Refondre l'onglet Négociations avec une structure de données commerciale complète
- Ajouter web search (Brave/Serper API) pour fournisseurs vérifiés
- Ajouter avertissement IA sur les données fournisseurs générées

I — Fonctionnalités à conserver

- Chat IA + réponses JSON structurées + export PDF
- Wizard fournisseurs 6 étapes (step par step UX)
- Génération de 11 documents fournisseur
- Analyse AO + génération 8 documents AO (avec correction PDF)
- Analyse logistique par territoire (DROM-COM)
- ComparisonMatrix + vue analytics
- Export CRM
- Historique sessions

J — Fonctionnalités à refaire

- Onglet Négociations : structure de données commerciale complète (12+ champs)
- Analyse AO : corriger la lecture des PDFs (multimodal)
- Rate limiting chat : migrer vers checkRateLimitAsync
- Isolation organisation : ajouter organization_id partout
- Persistance documents : Supabase Storage + table dédiée

K — Fonctionnalités à ajouter

- Web search fournisseurs réels (Brave API ou Serper)
- Claude tool_use pour actions DJAMA (créer fournisseur, créer commande)
- Table sourcing_documents (persistance documents générés)
- Table ai_usage_logs (suivi coûts, quotas, budget par org)
- Suivi scoring fournisseur persistant
- Intégration lecture stocks DJAMA
- Veille marchés publics structurée (BOAMP/TED scraping ou API)
- Onglet Marchés dédié avec opportunités filtrables
- Streaming des réponses IA (meilleure UX chat)
- Calcul TCO automatique
- Avertissement visible sur l'origine IA des données
- Protection prompt injection sur PDFs uploadés

ROADMAP — Phase 1 → Phase finale

Phase 1 — Sécurité & isolation (priorité immédiate)

- Ajouter organization_id à sourcing_sessions + sourcing_negotiations
- Nouvelle migration idempotente + RLS sur organization_id
- Migrer /sourcing/chat vers checkRateLimitAsync
- Ajouter protection prompt injection sur PDFs (copier pattern Scanner)
- Corriger /appel-offre/analyze : analyser réellement les PDFs en multimodal
- Ajouter avertissement visible 'données générées par IA' sur résultats fournisseurs

■ **Durée estimée : 2–3 jours. Aucune régression UI.**

Phase 2 — Persistance & documents (priorité haute)

- Créer table sourcing_documents (org_id, type, content, created_at)
- Sauvegarder automatiquement les documents générés dans Supabase Storage
- Implémenter la régénération avec chargement depuis la DB
- Lier sourcing_documents aux négociations (FK sourcing_negotiations)
- Ajouter ai_usage_logs pour suivi coûts

■ **Durée estimée : 3–5 jours. Données durables.**

Phase 3 — Négociations complètes (priorité haute)

- Refondre sourcing_negotiations : ajouter 12+ champs commerciaux
- Nouvelle migration : product, price_initial, price_target, quantity, moq, currency, incoterm, payment_terms, delivery_days
- Table sourcing_negotiation_messages (fil de discussion structuré)
- IA active dans les négociations (suggestions counter-offer, analyse BATNA)
- UI complète Négociations : formulaire riche + fil messages

■ Durée estimée : 5–7 jours. Fonctionnalité phare.

Phase 4 — Fournisseurs réels & intégrations DJAMA (priorité moyenne)

- Intégration web search (Brave API / Serper) pour fournisseurs vérifiés
- Lecture des fournisseurs ERP existants dans le contexte de recherche
- Connexion stocks DJAMA (vérification disponibilité)
- Export vers fournisseur ERP (pas seulement CRM)
- Claude tool_use : créer fournisseur, créer commande depuis le chat

■ Durée estimée : 7–10 jours. Données réelles.

Phase 5 — Intelligence avancée & UX finale

- Onglet Marchés dédié avec veille structurée (BOAMP, TED)
- Streaming des réponses IA (Server-Sent Events)
- Scoring fournisseur persistant et évolutif
- Calcul TCO automatique avec intégration logistique
- Tableau de bord Sourcing : KPIs, économies réalisées, performance fournisseurs
- Refonte UX mobile-first avec DJAMA Fluid/Glass Design System
- Intégrations Dépenses, Comptabilité, Contrats

■ Durée estimée : 10–14 jours. Produit complet.

Ce rapport a été produit par audit statique du code source uniquement. Aucun code n'a été modifié. Aucun test en conditions réelles n'a été exécuté. Pour valider les tests A–E, démarrer l'environnement de développement avec une clé Anthropic active.